

You already have a fairly strong feature set, but there is substantial redundancy. For XGBoost, redundancy is less problematic than for linear models, but highly correlated features can still increase variance, training time, and obscure interpretation.

First Observation: Target Leakage Risk

You currently create:

</> Python



▶ Run

```
df["Silver_ZAR"] = df["Silver_USD"] * df["USDZAR"]
```

and then include:

</> Python



▶ Run

```
lag_1, ..., lag_7
```

which are lagged **price levels**.

For return prediction, lagged price levels are usually much less informative than lagged returns and can introduce nonstationarity. I would generally drop all of the price-level lags:

</> Python



▶ Run

```
lag_1  
...  
lag_7
```

and keep only lagged returns.

Return Prediction

Target:

</> Python



▶ Run

```
target = log_returns.shift(-1)
```

Strong Features

These would be my first-choice predictors.

Silver Dynamics

</> Python



▶ Run

```
return_lag_1  
return_lag_2  
return_lag_3  
return_lag_4  
return_lag_5
```

You probably don't need all 7.

Returns tend to have very short memory.

Trend / Momentum

Keep:

</> Python



▶ Run

```
momentum_7  
momentum_14  
MA_ratio_short
```

Drop:

</> Python



▶ Run

```
MA_7  
MA_14  
MA_30
```

because:

$$MA_ratio_short = MA_7 / MA_{30}$$

already summarizes the information.

Precious Metals

Keep:

</> Python



▶ Run

```
gold_returns  
gold_silver_ratio
```

These often matter a lot.

Currency

Keep:

Python



Run

fx_returns

Arguably the most important feature for South African silver.

Drop:

Python



Run

fx_volatility

for return prediction.

It is more useful for volatility prediction.

Global Risk Sentiment

Keep:

Python



Run

sp500_returns

vix_changes

Interactions

Keep:

Python



Run

gold_fx_interaction

This captures simultaneous gold and currency shocks.

Potentially keep:

Python



Run

risk_interaction

but I'd test whether it improves CV performance.

Macro Variables

Keep:

</> Python



▶ Run

```
US_Interest_Rate
US_10Y_Yield
```

Potentially drop:

</> Python



▶ Run

```
US_CPI
US_Unemployment
```

Reason:

Daily silver returns rarely respond directly to slowly changing monthly macro levels.

Changes or surprises would be more informative than levels.

Recommended Return Model Features

</> Python



▶ Run

```
return_lag_1
return_lag_2
return_lag_3
return_lag_4
return_lag_5

momentum_7
momentum_14

MA_ratio_short

gold_returns
gold_silver_ratio

fx_returns

sp500_returns
vix_changes
```

```
gold_fx_interaction  
  
US_Interest_Rate  
US_10Y_Yield
```

≈ 15 predictors.

Volatility Prediction

Target:

Python



Run

```
target_vol = log_returns.shift(-1)**2
```

or preferably

Python



Run

```
target_vol = (  
    log_returns  
    .rolling(5)  
    .std()  
    .shift(-1)  
)
```

Strong Volatility Features

Volatility prediction is much easier than return prediction because volatility clusters.

Historical Volatility

Keep:

Python



Run

```
volatility_7  
volatility_14  
volatility_ratio
```

These are among your strongest predictors.

Lagged Returns

Instead of raw returns:

Create:

```
</> Python  
abs_return_lag_1
...
abs_return_lag_5
```

or

```
</> Python  
squared_return_lag_1
...
squared_return_lag_5
```

These are much better volatility predictors.

FX Volatility

Keep:

```
</> Python  
fx_volatility
```

Very useful.

VIX

Keep:

```
</> Python  
VIX
vix_changes
```

The VIX is essentially a volatility predictor.

Gold

Keep:

Python



Run

```
gold_returns
```

Gold and silver volatility often move together.

Better yet, create:

Python



Run

```
gold_volatility
```

using a rolling standard deviation.

Equity Markets

Keep:

Python



Run

```
sp500_returns
```

Better:

Python



Run

```
sp500_volatility
```

Create a rolling volatility measure.

Yield Variables

Keep:

Python



Run

```
US_10Y_Yield
```

Interest-rate uncertainty influences precious metals.

Recommended Volatility Model Features

</> Python



▶ Run

```
volatility_7
volatility_14
volatility_ratio

fx_volatility

VIX
vix_changes

gold_returns
gold_silver_ratio

sp500_returns

US_10Y_Yield

abs_return_lag_1
abs_return_lag_2
abs_return_lag_3
abs_return_lag_4
abs_return_lag_5
```

Features I Would Remove Entirely

These are either redundant or unlikely to add much predictive value.

</> Python



▶ Run

```
lag_1
lag_2
lag_3
lag_4
lag_5
lag_6
lag_7
```

Reason: price levels are nonstationary and mostly dominated by return information.

</> Python



▶ Run

```
MA_7
MA_14
MA_30
```

Reason: summarized by MA_ratio_short.

Python



Run

```
returns
```

Reason: log_returns contains essentially the same information.

Potentially:

Python



Run

```
US_CPI
US_Unemployment
```

because they update monthly and remain constant across many trading days.

Additional Features Worth Adding

For a strong ARMA-GARCH vs XGBoost comparison, I would add:

Python



Run

```
df["realized_vol_21"] = (
    df["log_returns"]
    .rolling(21)
    .std()
)
```

Python



Run

```
df["realized_vol_63"] = (
    df["log_returns"]
    .rolling(63)
    .std()
)
```

Python



Run

```
df["gold_vol_21"] = (
    df["gold_returns"]
    .rolling(21)
    .std()
)
```

Python



Run

```
df["sp500_vol_21"] = (  
    df["sp500_returns"]  
    .rolling(21)  
    .std()  
)
```

These longer-horizon volatility measures often become some of the most important features when predicting next-day silver volatility.

A practical feature set

If I were conducting a master's thesis comparison of ARMA-GARCH versus XGBoost for South African silver, I'd start with roughly:

Return model (15 features):

- return_lag_1–5
- momentum_7
- momentum_14
- MA_ratio_short
- gold_returns
- gold_silver_ratio
- fx_returns
- sp500_returns
- vix_changes
- gold_fx_interaction
- US_Interest_Rate
- US_10Y_Yield

Volatility model (15–20 features):

- volatility_7
- volatility_14
- volatility_ratio
- realized_vol_21
- realized_vol_63
- fx_volatility
- VIX
- vix_changes
- gold_vol_21
- sp500_vol_21
- abs_return_lag_1–5
- US_10Y_Yield

This would give you a defensible, non-redundant feature set that captures commodity dynamics, exchange-rate effects, market sentiment, and volatility clustering while remaining compact enough for reliable cross-validation.